

Bubble Sort – Optimized

Problem Statement

Given an array `arr[]`, sort the array using the **Bubble Sort algorithm**. The array should be sorted in ascending order.

Examples

Example 1

Input: `arr[] = [4, 1, 3, 9, 7]`
Output: `[1, 3, 4, 7, 9]`

Explanation: After sorting the array in ascending order, the result is `[1, 3, 4, 7, 9]`.

Example 2

Input: `arr[] = [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]`
Output: `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`

Explanation: The elements are arranged in ascending order using Bubble Sort.

Example 3

Input: `arr[] = [1, 2, 3, 4, 5]`
Output: `[1, 2, 3, 4, 5]`

Explanation: The array is already sorted, so no changes are required.

Approach

Bubble Sort repeatedly compares adjacent elements. If the left element is greater than the right element, the two elements are swapped. After every complete pass, the largest unsorted element moves to the end of the array.

The optimized version uses a boolean variable **swapped**. If no swap occurs during a complete pass, the array is already sorted, so the algorithm stops immediately. This reduces the best-case time complexity from $O(n^2)$ to $O(n)$.

Algorithm

1. Set `n = arr.length`.
2. Run an outer loop from `i = 0` to `n - 2`.
3. Set `swapped = false` at the beginning of each pass.
4. Compare adjacent elements from index `0` to `n - i - 2`.
5. If `arr[j] > arr[j + 1]`, swap the two elements and set `swapped = true`.
6. After the inner loop, check `swapped`.
7. If `swapped` is false, break because the array is already sorted.
8. Continue until the array is sorted.

Java Implementation

```
class Solution {  
    public void bubbleSort(int[] arr) {
```

```

int n = arr.length;
for (int i = 0; i < n - 1; i++) {
    boolean swapped = false;
    for (int j = 0; j < n - i - 1; j++) {
        if (arr[j] > arr[j + 1]) {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;

            swapped = true;
        }
    }
    if (!swapped) {
        break;
    }
}
}

```

Dry Run

Consider:

```
arr = [4, 1, 3, 9, 7]
```

Pass	Comparison	Result
1	4 > 1 → swap	[1, 4, 3, 9, 7]
1	4 > 3 → swap	[1, 3, 4, 9, 7]
1	4 < 9 → no swap	[1, 3, 4, 9, 7]
1	9 > 7 → swap	[1, 3, 4, 7, 9]
2	1 < 3, 3 < 4, 4 < 7	No swaps

After Pass 1, 9 reaches its correct position. During Pass 2, no swaps occur, so swapped remains false and the algorithm terminates early.

Final Output: [1, 3, 4, 7, 9]

Why $n - i - 1$?

After every pass, the largest remaining unsorted element is placed at the end. Therefore, the already sorted suffix does not need to be compared again.

```

Pass 1 → largest element fixed
Pass 2 → second largest element fixed
Pass 3 → third largest element fixed

```

Why swapped?

The swapped variable detects whether any exchange happened during a pass. For an already sorted array such as [1, 2, 3, 4, 5], no swap is required. Therefore swapped remains false and the algorithm stops after one pass.

Complexity Analysis

Case	Time Complexity
------	-----------------

Best Case	$O(n)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space Complexity	$O(1)$

The best case is $O(n)$ because the optimized algorithm stops when a complete pass makes no swaps. The algorithm sorts in-place, so extra space is $O(1)$.

Alternative Approach

Java provides built-in sorting methods such as `Arrays.sort(arr)`. However, this problem specifically asks for Bubble Sort, so the algorithm must be implemented manually.

Key Interview Takeaway

- Bubble Sort compares adjacent elements.
- If `arr[j] > arr[j + 1]`, the elements are swapped.
- After each pass, the largest unsorted element moves to the end.
- `n - i - 1` avoids comparisons with the sorted suffix.
- The swapped flag provides early termination.
- Best Case: $O(n)$ with optimization.
- Average and Worst Case: $O(n^2)$.
- Space Complexity: $O(1)$.
- Bubble Sort is an in-place sorting algorithm.

Interview Takeaway

The main optimization in Bubble Sort is the swapped flag. It avoids unnecessary passes when the array is already sorted, making the best-case time complexity $O(n)$ while keeping the algorithm in-place with $O(1)$ extra space.